Detecting LLM-Written Code on AIGCodeSet: Tuned Code Encoders versus N-gram Baselines

Andreas Bartsiokas - LT2326 Machine Learning - October 2026

Abstract

I study whether code written by an LLM can be told apart from human code, and which LLM wrote it, on AIGCodeSet. I compare word and character n-gram logistic regression with fine-tuned code encoders (CodeBERT, GraphCodeBERT, UniXcoder) on four task variants, using problem-grouped splits and a test set touched once. A tuned character n-gram baseline outperforms CodeBERT and GraphCodeBERT. A tuned UniXcoder outperforms the baseline on all four variants, though the margin is statistically clear only for binary detection. Averaging six seeds and mixing with the baseline improve the scores further. Errors are concentrated in two places: code that an LLM *edited* from a human submission (detected 27-54% of the time, against 94-98% for code written from scratch) and Gemini, whose from-scratch code is often classified as human. Identifier names are the strongest single cue I could isolate.

1. Introduction

LLM-generated code is hard to separate from student or competition code by inspection. I treat this as an empirical question with three parts: (i) How well do cheap lexical baselines and fine-tuned code encoders do under a split that never shares a programming problem between train and test. (ii) Which hyperparameters and cues actually matter. (iii) Where do the models fail. The study is my own evaluation, not a replication: the dataset paper [1] reports binary detection with classical classifiers on a random split, so my numbers are not comparable to theirs. Recent work has shown that detection performance depends strongly on the evaluation setting and degrades under more realistic conditions such as unseen generators, hybrid authorship, edited code, and distribution shift [6,7].

2. Data

AIGCodeSet [1] pairs human solutions from CodeNet [2] with code from three LLMs (CodeLlama 34B, Codestral 22B, Gemini 1.5 Flash). An LLM either writes a solution from scratch (*generate*) or *edits* a human submission (fixing a runtime error or a wrong output). This distinction is important because both AIGCodeSet and MultiAIGCD report substantially lower detection performance for AI-edited code than for code generated from scratch [1,7]. I keep 7,531 samples after deduplication and split by problem: train 5,267, dev 739, test 1,525, with no problem shared across splits. Files whose identical code appears under both labels (478) are removed from every split, leaving 681 dev and 1,419 test files. This makes the splits slightly easier than the raw data.

I define four variants: `binary` (human vs AI) and `multiclass` (human, GEMINI, LLAMA, CODESTRAL) on all data, and `binary_generate` and `multiclass_generate`, which drop edited code. Test sizes are in Table 1. Models see only a normalized copy of the code (trailing spaces, tabs, leading blank space and final-newline differences removed). A raw-format audit had found such tells (for example 21% of LLAMA files start with whitespace), and no metadata column reaches a model.

Table 1. Test set sizes.

Variant	Human	AI (per class)
binary / multiclass	905	514 (GEMINI 179, LLAMA 146, CODESTRAL 189)
binary_generate / multiclass_generate	905	186 (GEMINI 62, LLAMA 61, CODESTRAL 63)

3. Methods

Baselines. TF-IDF with logistic regression on word or character n-grams. For the character model I chose n-grams (1,6), C=10 and 300k features by 7-fold cross-validation on the training folds only.

Encoders. CodeBERT [3], GraphCodeBERT [4] and UniXcoder [5], fully fine-tuned with AdamW, a linear schedule with 10% warmup, weight decay 0.01, batch size 16 and learning rate 2e-5 for 4 epochs, no class weights. Half of the files exceed 128 tokens, so I use 256 tokens for the final model. The checkpoint with the best dev macro-F1 at an epoch end is kept. All models train on the train split only.

Tuning protocol. I tuned on the two generate variants with seeds 0-2, adopting a change only if the mean dev gain was at least .01 on both variants and no more than one of six paired seed comparisons was negative. Dev has only 93 AI files in `binary_generate` and 31 per class in `multiclass_generate`, so differences below about .03 are treated as ties. The final models use fresh seeds 3-8, and each run is evaluated on test once.

Ensembles and statistics. A *seed ensemble* averages the class probabilities of the six UniXcoder runs. A *mix* averages the ensemble with the character baseline at weight 0.5 (fixed, not tuned). The gap to the baseline is tested with a paired bootstrap over problems (1,000 resamples), so files from one problem stay together. The metric is macro-F1.

4. Results

4.1 Final test results

Table 2. Test macro-F1. UniXcoder single seed is mean ± std over 6 seeds. The intervals are 95% bootstrap intervals for the difference from the character baseline.

Variant	char LogReg	word LogReg	UniXcoder, 1 seed	Seed ensemble	Mix 0.5	1 seed - char	Ensemble - char
binary	.743	.658	.772 ± .002	.778	**.783**	+.029 [+.011, +.048]	+.034 [+.015, +.054]
multiclass	.554	.477	.571 ± .026	.595	**.614**	+.017 [-.016, +.052]	+.041 [+.003, +.077]
binary_generate	.923	.885	.939 ± .008	.943	**.948**	+.016 [-.003, +.035]	+.020 [+.001, +.041]
multiclass_generate	.808	.772	.837 ± .016	.839	**.867**	+.029 [-.006, +.066]	+.031 [-.008, +.070]

UniXcoder is ahead of the character baseline on all four variants. A single model is significantly ahead only on `binary`. The seed ensemble is significantly ahead on three of four (`multiclass` and `binary_generate` only barely). The mix is highest everywhere, but on dev it had been a tie with the ensemble and it was added to the analysis after seeing dev, so I report it as an extra row without an interval. Dev scores on the fresh seeds (.923 and .818 on the generate variants) are close to those from tuning (.926 and .829), so the tuning inflated dev by little.

4.2 What mattered in training

At 128 tokens on dev, the pretrained checkpoint made the largest difference: UniXcoder .896 / .778, GraphCodeBERT .869 / .739, CodeBERT .834 / .712 (`binary_generate` / `multiclass_generate`), and UniXcoder beat GraphCodeBERT in all six paired seed comparisons. The importance of checkpoint choice is consistent with recent benchmarks, which report substantial performance differences between code encoders and modern transformer architectures on AI-code detection tasks [6]. CodeBERT and GraphCodeBERT were

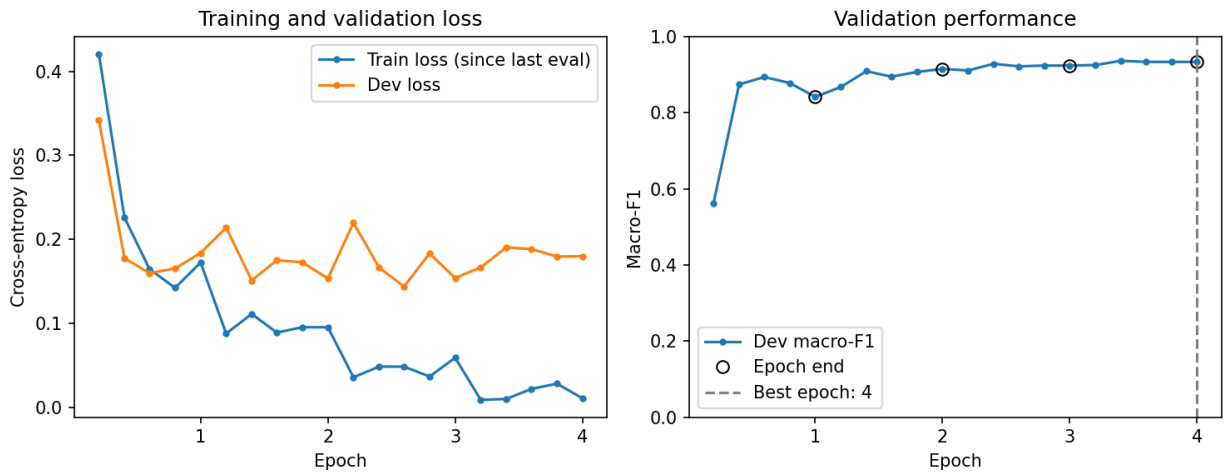


Figure 1: Example training curve of one of the seeds for UniXcoder (seed 5), `binary_generate`, dev macro-F1 and loss, 5 evaluations per epoch.

not evaluated on test. Inverse-frequency class weights hurt multiclass (CodeBERT `multiclass` .404 against .515 without weights), because they flooded the human-heavy dev set with false AI predictions.

Table 3. UniXcoder, 256 tokens, dev macro-F1, seeds 0-2.

Setting	binary_generate	multiclass_generate
dropout 0.1 (starting point)	.906	.798
dropout 0.0	**.926**	**.829**
dropout 0.2	.862	.751
dropout 0.0, 6 epochs	.930	.815
dropout 0.0, batch 8 / 32	.924 / .921	.819 / .803
dropout 0.0, weight decay 0.1	.921	.821
dropout 0.1: 384 tokens / lr 3e-5 / lr 1e-5	.911 / .904 / .901	.774 / .793 / .780

Only removing dropout produced a consistent gain across all six paired seeds. With 4-5k training files the model memorizes the training set (train loss falls to about .003) while dev F1 stays flat, and dev F1 itself swings by .02-.04 between neighbouring epochs on the small multiclass dev set.

4.3 Which cues carry the signal

I ablated the input of the character baseline (7-fold CV on the training folds, generate variants).

Table 4. Macro-F1 mean over folds.

Input	binary_generate	multiclass_generate
full code	.909	.791
layout removed	.907	.778
comments removed	.902	.787
identifier names masked	.882	.765

Masking identifiers costs the most (about .026). Layout and comments are within fold noise (std .02-.03). Most of the signal survives name masking, suggesting no single cue explains detection. These ablations are

not additive and say how much a cue contributes, not in which direction it points. I cannot say from my experiments whether the encoders use the same cues.

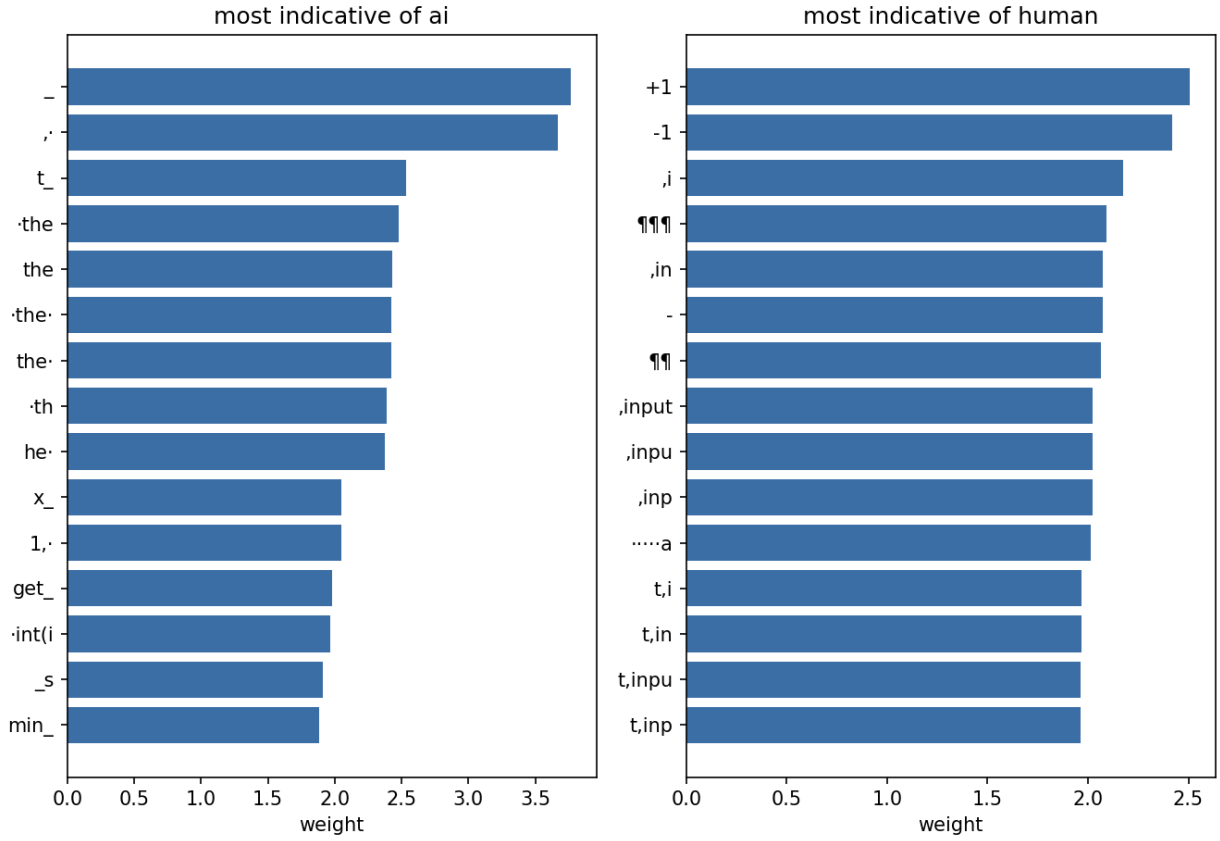


Figure 2: top character n-grams of the char LogReg, `binary_generate`

4.4 Where the models fail

Table 5. Share of files detected as AI, by true source, `binary`, test.

True source	n	UniXcoder ensemble	char LogReg
Human (false positives)	905	9.9%	8.7%
LLAMA, generate	61	98.4%	98.4%
CODESTRAL, generate	63	98.4%	92.1%
GEMINI, generate	62	93.5%	90.3%
LLAMA, edit	85	54.1%	43.5%
CODESTRAL, edit	126	54.0%	35.7%
GEMINI, edit	117	27.4%	22.2%

Edited code is the main source of error: it is 328 of the 514 AI files in `binary`, and it is detected far less often than code written from scratch, which explains the gap between `binary` (.78) and `binary_generate` (.94). This agrees with both AIGCodeSet and MultiAIGCD, where AI-edited code is consistently harder to detect than code generated from scratch [1,7]. In `multiclass_generate`, 37% of GEMINI files are predicted Human, while confusions between LLMs are small (GEMINI to LLAMA 2%, to CODESTRAL 5%) (Figure 3). Gemini is therefore hard because it looks human to the models, not because it resembles the other two

LLMs. When code is written from scratch, attribution works: F1 .91 (LLAMA) and .82 (CODESTRAL) for the ensemble, .96 and .86 for the mix. The original AIGCodeSet paper [1] also reports lower recall for Gemini-generated code than for CodeLlama- or Codestral-generated code. On the full data about 1 in 10 human files is flagged as AI.

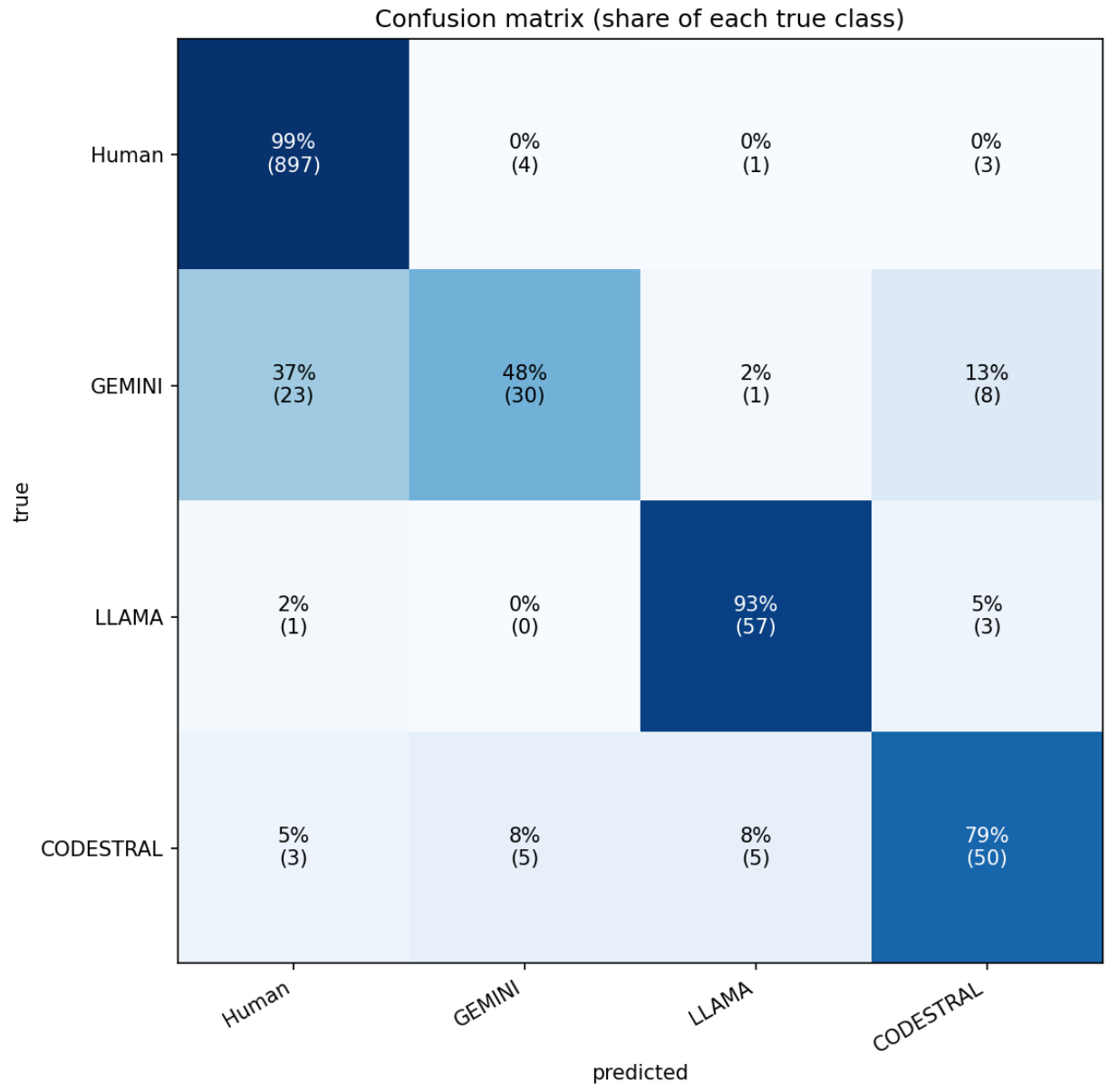


Figure 3: confusion matrix, UniXcoder, `multiclass_generate`, test

5. Discussion

Supported by my results. (1) A tuned character n-gram model is a strong baseline that CodeBERT and GraphCodeBERT do not reach under a problem-grouped split. (2) UniXcoder, with dropout removed, is at least as good on every variant and clearly better on `binary`. (3) Averaging seeds gives a small, reliable gain, and combining the encoder with the n-gram model gave the best test scores. (4) Edited code and Gemini are the hardest cases. This closely matches AIGCodeSet, where performance varies sharply across generation

scenarios and Gemini outputs are generally harder to identify than CodeLlama or Codestral outputs [1]. It also agrees with MultiAIGCD, which reports substantially lower performance for AI-edited code than for from-scratch generation across multiple languages and detector architectures [7].

Speculation, not tested. UniXcoder may do better than CodeBERT because of its pretraining objectives, but I did not run a pretraining ablation. Dropout may hurt because the training set is small, I only observed the effect. The mix may help because the two models make different mistakes, which I did not measure directly. Gemini may write code closer to human style, but I did not analyse its style. Recent benchmark work reports that detector performance degrades under distribution shift, unseen generators, and more realistic authorship settings, suggesting that current results are often sensitive to the training distribution [6]. MultiAIGCD likewise finds substantial cross-model and cross-language performance drops, highlighting the importance of diverse evaluation settings [7].

6. Limitations

- Dev is small, so tuning decisions on the multiclass variants are noisy. About 20 configurations were compared on dev, fresh seeds show the resulting optimism is small (.003-.011).
- `binary` and `multiclass` reuse a configuration tuned on the generate variants only.
- Test sets are small for the LLM classes (62 files each in the generate variants), so several intervals include zero.
- The mix was added after seeing dev, and its weight was not tuned.
- Single data source, no out-of-distribution evaluation and no human-only external test (such as MBPP). Conflict rows were removed from all splits.
- I did not run ModernBERT (library too old) or a larger code LLM with LoRA.
- My study uses one dataset and one programming language. Recent benchmark work shows that detector performance can vary substantially across programming languages, generators, and evaluation scenarios [6,7].

7. Conclusion

On AIGCodeSet, a tuned UniXcoder with dropout 0.0 and 256 tokens is consistently ahead of the best n-gram baseline, with a clear margin on binary detection and smaller, mostly non-significant margins on the other variants. Seed averaging and mixing with the baseline add a little. The remaining difficulty lies in edited code and in Gemini. The most useful next experiments would be an external human-only test set for false positives and a split that holds out an LLM to test generalization. Consistent with recent benchmark results, the remaining challenge is not detecting obvious AI-written code but handling edited or hybrid code and maintaining performance under more realistic evaluation conditions [6,7].

References

- [1] Demirok, Kutlu. AIGCodeSet: A New Annotated Dataset for AI Generated Code Detection
- [2] R. Puri et al. Project CodeNet. 2021.
- [3] Z. Feng et al. CodeBERT. Findings of EMNLP, 2020.
- [4] D. Guo et al. GraphCodeBERT. ICLR, 2021.
- [5] D. Guo et al. UniXcoder. ACL, 2022.
- [6] Orel et al. AICD Bench: A Challenging Benchmark for AI-Generated Code Detection. EACL 2026.
- [7] Demirok, Kutlu, Mergen. MultiAIGCD: a comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios. Neural Computing and Applications, 2026.